

LIST --> Mutable

```
In [22]: l=[]  
l
```

```
Out[22]: []
```

```
In [23]: type(l)
```

```
Out[23]: list
```

```
In [24]: len(l)
```

```
Out[24]: 0
```

Append()

```
In [25]: l.append(10) #append()-> add the value at the end of the list  
l
```

```
Out[25]: [10]
```

```
In [26]: len(l)
```

```
Out[26]: 1
```

```
In [27]: l.append(10,20) #only one argument can be appended  
l
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[27], line 1  
----> 1 l.append(10,20) #only one argument can be appended  
      2 l  
  
TypeError: list.append() takes exactly one argument (2 given)
```

```
In [28]: l1=[10]  
l1
```

```
Out[28]: [10]
```

```
In [29]: l1.append(20)  
l1.append(30)  
l1.append(40)  
l1.append(50)  
l1
```

```
Out[29]: [10, 20, 30, 40, 50]
```

```
In [30]: len(l1)
```

```
Out[30]: 5
```

```
In [31]: l1.append(10) #Duplicate is allowed  
l1
```

```
Out[31]: [10, 20, 30, 40, 50, 10]
```

```
In [32]: l2=[10,2.8,"mousumi",12+2j,True] #In list we can assign different types of data  
l2
```

```
Out[32]: [10, 2.8, 'mousumi', (12+2j), True]
```

```
In [33]: l2.append([1,2,3]) #nested list --> list inside another list is nested list  
l2
```

```
Out[33]: [10, 2.8, 'mousumi', (12+2j), True, [1, 2, 3]]
```

copy()

```
In [34]: print(l1)  
print(l2)
```

```
[10, 20, 30, 40, 50, 10]  
[10, 2.8, 'mousumi', (12+2j), True, [1, 2, 3]]
```

```
In [35]: l1
```

```
Out[35]: [10, 20, 30, 40, 50, 10]
```

```
In [36]: l3=l1.copy() #to copy one list to another list  
l3
```

```
Out[36]: [10, 20, 30, 40, 50, 10]
```

```
In [37]: print(l1)  
print(l2)  
print(l3)
```

```
[10, 20, 30, 40, 50, 10]  
[10, 2.8, 'mousumi', (12+2j), True, [1, 2, 3]]  
[10, 20, 30, 40, 50, 10]
```

```
In [38]: l1==l3
```

```
Out[38]: True
```

```
In [39]: len(l1)==len(l3)
```

```
Out[39]: True
```

```
In [40]: id(l1)==id(l3) #have same value but the id is not same
```

```
Out[40]: False
```

```
In [41]: print(id(l1)) #both have different id  
print(id(l3))
```

1232003825984
1232003097600

clear()

```
In [42]: print(l1)
         print(l2)
         print(l3)
```

```
[10, 20, 30, 40, 50, 10]
[10, 2.8, 'mousumi', (12+2j), True, [1, 2, 3]]
[10, 20, 30, 40, 50, 10]
```

```
In [43]: l3.clear() #it clear all the value and gives empty list
```

```
In [44]: print(l1)
         print(l2)
         print(l3)
```

```
[10, 20, 30, 40, 50, 10]
[10, 2.8, 'mousumi', (12+2j), True, [1, 2, 3]]
[]
```

```
In [45]: del l3 #to delete any variable we use delete
```

```
In [46]: l3
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[46], line 1
----> 1 l3

NameError: name 'l3' is not defined
```

```
In [47]: print(l1)
         print(l2)
```

```
[10, 20, 30, 40, 50, 10]
[10, 2.8, 'mousumi', (12+2j), True, [1, 2, 3]]
```

count()

```
In [48]: l1.count(20) #count the occurances of variable
```

```
Out[48]: 1
```

```
In [49]: l1.count(10)
```

```
Out[49]: 2
```

extend()

```
In [50]: print(l1)
         print(l2)
```

```
[10, 20, 30, 40, 50, 10]  
[10, 2.8, 'mousumi', (12+2j), True, [1, 2, 3]]
```

```
In [51]: l1.extend(l2) #we can merge the list  
l1
```

```
Out[51]: [10, 20, 30, 40, 50, 10, 10, 2.8, 'mousumi', (12+2j), True, [1, 2, 3]]
```

```
In [52]: print(len(l1))  
print(len(l2))
```

```
12  
6
```

index()

```
In [53]: print(l1)  
print(l2)
```

```
[10, 20, 30, 40, 50, 10, 10, 2.8, 'mousumi', (12+2j), True, [1, 2, 3]]  
[10, 2.8, 'mousumi', (12+2j), True, [1, 2, 3]]
```

```
In [54]: l1.index(20) #indexing is allowed in list
```

```
Out[54]: 1
```

```
In [55]: l2.index(12+2j)
```

```
Out[55]: 3
```

```
In [56]: l2.index("mousumi")
```

```
Out[56]: 2
```

```
In [57]: l2[:]
```

```
Out[57]: [10, 2.8, 'mousumi', (12+2j), True, [1, 2, 3]]
```

```
In [58]: l2[5]
```

```
Out[58]: [1, 2, 3]
```

```
In [59]: l2[6]
```

```
-----  
IndexError                                Traceback (most recent call last)  
Cell In[59], line 1  
----> 1 l2[6]  
  
IndexError: list index out of range
```

```
In [60]: l2[:3]
```

```
Out[60]: [10, 2.8, 'mousumi']
```

```
In [61]: l2[3:]
```

Out[61]: [(12+2j), True, [1, 2, 3]]

In [62]: 12

Out[62]: [10, 2.8, 'mousumi', (12+2j), True, [1, 2, 3]]

In [63]: 12[0:5:2]

Out[63]: [10, 'mousumi', True]

In [64]: 12[-3]

Out[64]: (12+2j)

In [65]: 12[-3:]

Out[65]: [(12+2j), True, [1, 2, 3]]

In [66]: 12[::-1]

Out[66]: [[1, 2, 3], True, (12+2j), 'mousumi', 2.8, 10]

In [67]: 12[::-2]

Out[67]: [[1, 2, 3], (12+2j), 2.8]

In [68]: 12

Out[68]: [10, 2.8, 'mousumi', (12+2j), True, [1, 2, 3]]

In [69]: 12[2]

Out[69]: 'mousumi'

In [76]: print(12[2][0]) *#nested index--> index within another index*
 print(12[2][1])
 print(12[2][2])
 print(12[2][3])
 print(12[2][4])
 print(12[2][5])
 print(12[2][6])
 print(12[2][7])

m
o
u
s
u
m
i

```

-----
IndexError                                Traceback (most recent call last)
Cell In[76], line 8
      6 print(l2[2][5])
      7 print(l2[2][6])
----> 8 print(l2[2][7])

IndexError: string index out of range

```

In [71]: 12

Out[71]: [10, 2.8, 'mousumi', (12+2j), True, [1, 2, 3]]

In [72]: 12[0]=100 *#mutable-->changable (list is mutable)*
12

Out[72]: [100, 2.8, 'mousumi', (12+2j), True, [1, 2, 3]]

In [73]: 12

Out[73]: [100, 2.8, 'mousumi', (12+2j), True, [1, 2, 3]]

17-08-26

insert

In [74]: 13=[10,20,30]
13

Out[74]: [10, 20, 30]

In [75]: 13[0]=100
13

Out[75]: [100, 20, 30]

In [77]: 13.insert(2,25)
13

Out[77]: [100, 20, 25, 30]

In [78]: 13.insert(2,[1,2,3])
13

Out[78]: [100, 20, [1, 2, 3], 25, 30]

pop() --> remove value based on indexing

In [79]: 13.pop()

Out[79]: 30

In [80]: 13

```
Out[80]: [100, 20, [1, 2, 3], 25]
```

```
In [81]: 12
```

```
Out[81]: [100, 2.8, 'mousumi', (12+2j), True, [1, 2, 3]]
```

```
In [82]: 12.pop()
```

```
Out[82]: [1, 2, 3]
```

```
In [83]: 12.pop(-2)
```

```
Out[83]: (12+2j)
```

```
In [84]: 12
```

```
Out[84]: [100, 2.8, 'mousumi', True]
```

remove() --> direct remove value not indexing

```
In [90]: 12.remove(2.8)
```

```
In [91]: 12
```

```
Out[91]: [100, 'mousumi']
```

reverse()

```
In [92]: print(l1)
         print(l2)
         print(l3)
```

```
[10, 20, 30, 40, 50, 10, 10, 2.8, 'mousumi', (12+2j), True, [1, 2, 3]]
[100, 'mousumi']
[100, 20, [1, 2, 3], 25]
```

```
In [95]: 12.reverse()
         12
```

```
Out[95]: ['mousumi', 100]
```

```
In [96]: 13[::-1]
```

```
Out[96]: [25, [1, 2, 3], 20, 100]
```

sort() --> by default it gives ascending sorting

--> it only works when we have similar datatype

```
In [97]: 14=[3,300,23,200,1000,10]
         14
```

Out[97]: [3, 300, 23, 200, 1000, 10]

In [98]: `l4.sort()` *#bydefault it is reverse=False i.e, descending sort*
14

Out[98]: [3, 10, 23, 200, 300, 1000]

In [99]: `l4.sort( reverse=True)`
14

Out[99]: [1000, 300, 200, 23, 10, 3]

In [100... `l3.sort()`
13

```
-----
TypeError                                Traceback (most recent call last)
Cell In[100], line 1
----> 1 l3.sort()
      2 l3

TypeError: '<' not supported between instances of 'list' and 'int'
```

list membership

In [101... 14

Out[101... [1000, 300, 200, 23, 10, 3]

In []:

In [102... `10 in l4`

Out[102... True

In [103... `100 in l4`

Out[103... False

All and Any --> in list if we have 0 value ALL will print false , else true

In [104... 14

Out[104... [1000, 300, 200, 23, 10, 3]

In [105... `all(l4)`

Out[105... True

In [106... `any(l4)`

Out[106... True


```
In [107... 14.append(0)
14
```

```
Out[107... [1000, 300, 200, 23, 10, 3, 0]
```

```
In [108... all(14)
```

```
Out[108... False
```

```
In [109... any(14)
```

```
Out[109... True
```

PRACTICE (17-08-26)

strip --> Removes whitespace

lstrip --> Removes whitespace from left side only

rstrip --> Removes whitespace from right side only

```
In [10]: txt = "  abc def ghi  "
txt.lstrip()
```

```
Out[10]: 'abc def ghi  '
```

```
In [6]: txt = "  abc def ghi  "
txt.strip()
```

```
Out[6]: 'abc def ghi'
```

```
In [7]: txt = "  abc def ghi  "
txt.rstrip()
```

```
Out[7]: '  abc def ghi'
```

escape character (\)

```
In [8]: #Using double quotes in the string is not allowed.
mystr = "My favourite TV Series is "Game of Thrones""
```

```
Cell In[8], line 2
    mystr = "My favourite TV Series is "Game of Thrones""
                                   ^
SyntaxError: invalid syntax
```

```
In [9]: mystr = "My favourite series is \"Game of Thrones\""
```

```
print(mystr)
```

My favourite series is "Game of Thrones"

List question practice

```
In [11]: list1 = [] # Empty List
```

```
In [12]: print(type(list1))
```

```
<class 'list'>
```

```
In [13]: list2 = [10,30,60] # List of integers numbers
```

```
In [14]: list3 = [10.77,30.66,60.89] # List of float numbers
```

```
In [15]: list4 = ['one','two' , "three"] # List of strings
```

```
In [16]: list5 = ['Asif', 25 ,[50, 100],[150, 90]] # Nested Lists
```

```
In [17]: list6 = [100, 'Asif', 17.765] # List of mixed data types
```

```
In [18]: list7 = ['Asif', 25 ,[50, 100],[150, 90] , {'John' , 'David'}]
```

```
In [19]: len(list6) #Length of List
```

```
Out[19]: 3
```

list indexing

```
In [20]: list2[0] # Retrieve first element of the List
```

```
Out[20]: 10
```

```
In [21]: list4[0] # Retrieve first element of the List
```

```
Out[21]: 'one'
```

```
In [22]: list4[0][0] # Nested indexing - Access the first character of the first list ele
```

```
Out[22]: 'o'
```

```
In [23]: list4[-1] # Last item of the List
```

```
Out[23]: 'three'
```

```
In [24]: list5[-1] # Last item of the List
```

```
Out[24]: [150, 90]
```

list slicing

```
In [25]: mylist = ['one' , 'two' , 'three' , 'four' , 'five' , 'six' , 'seven' , 'eight']

In [26]: mylist[0:3] # Return all items from 0th to 3rd index location excluding the item
Out[26]: ['one', 'two', 'three']

In [27]: mylist[2:5] # List all items from 2nd to 5th index location excluding the item a
Out[27]: ['three', 'four', 'five']

In [28]: mylist[:3] # Return first three items
Out[28]: ['one', 'two', 'three']

In [29]: mylist[:2] # Return first two items
Out[29]: ['one', 'two']

In [30]: mylist[-3:] # Return last three items
Out[30]: ['six', 'seven', 'eight']

In [31]: mylist[-2:] # Return last two items
Out[31]: ['seven', 'eight']

In [32]: mylist[-1] # Return last item of the list
Out[32]: 'eight'

In [33]: mylist[:] # Return whole list
Out[33]: ['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight']
```

Add,remove and change items

```
In [34]: mylist
Out[34]: ['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight']

In [35]: mylist.append('nine') # Add an item to the end of the list
mylist
Out[35]: ['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine']

In [36]: mylist.insert(9,'ten') # Add item at index location 9
mylist
Out[36]: ['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine', 'ten']

In [37]: mylist.insert(1,'ONE') # Add item at index location 1
mylist
```

```
Out[37]: ['one',  
          'ONE',  
          'two',  
          'three',  
          'four',  
          'five',  
          'six',  
          'seven',  
          'eight',  
          'nine',  
          'ten']
```

```
In [38]: mylist.remove('ONE') # Remove item "ONE"  
mylist
```

```
Out[38]: ['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine', 'ten']
```

```
In [39]: mylist.pop() # Remove last item of the list  
mylist
```

```
Out[39]: ['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine']
```

```
In [40]: mylist.pop(8) # Remove item at index location 8  
mylist
```

```
Out[40]: ['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight']
```

```
In [41]: del mylist[7] # Remove item at index location 7  
mylist
```

```
Out[41]: ['one', 'two', 'three', 'four', 'five', 'six', 'seven']
```

```
In [42]: #changing value of the string
```

```
mylist[0] = 1  
mylist[1] = 2  
mylist[2] = 3  
mylist
```

```
Out[42]: [1, 2, 3, 'four', 'five', 'six', 'seven']
```

```
In [43]: mylist.clear() # Empty List / Delete all items in the list  
mylist
```

```
Out[43]: []
```

```
In [44]: del mylist # Delete the whole list  
mylist
```

```
-----  
NameError                                Traceback (most recent call last)  
Cell In[44], line 2  
      1 del mylist # Delete the whole list  
----> 2 mylist  
NameError: name 'mylist' is not defined
```

copy list

```
In [45]: mylist = ['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine']

In [88]: mylist1 = mylist # Create a new reference "mylist1" (= --> same box)

In [47]: id(mylist) , id(mylist1) # The address of both mylist & mylist1 will be the same

Out[47]: (2113068301632, 2113068301632)

In [89]: mylist2 = mylist.copy() # Create a copy of the list (.copy -> new box)

In [49]: id(mylist2) # The address of mylist2 will be different from mylist because mylis

Out[49]: 2113074732864

In [51]: mylist[0] = 1

In [52]: mylist

Out[52]: [1, 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine']

In [53]: mylist1 # mylist1 will be also impacted as it is pointing to the same list

Out[53]: [1, 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine']

In [90]: mylist2 # Copy of list won't be impacted due to changes made on the original lis

Out[90]: [1, 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine']
```

join list

```
In [55]: list1 = ['one', 'two', 'three', 'four']
         list2 = ['five', 'six', 'seven', 'eight']

In [56]: list3 = list1 + list2 # Join two lists by '+' operator
         list3

Out[56]: ['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight']

In [57]: list1.extend(list2) #Append list2 with list1
         list1

Out[57]: ['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight']
```

list membership

```
In [58]: list1

Out[58]: ['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight']
```

```
In [59]: 'one' in list1 # Check if 'one' exist in the list
```

```
Out[59]: True
```

```
In [60]: 'ten' in list1 # Check if 'ten' exist in the list
```

```
Out[60]: False
```

```
In [63]: if 'three' in list1: # Check if 'three' exist in the list
          print('Three is present in the list')
        else:
          print('Three is not present in the list')
```

Three is present in the list

```
In [64]: if 'eleven' in list1: # Check if 'eleven' exist in the list
          print('eleven is present in the list')
        else:
          print('eleven is not present in the list')
```

eleven is not present in the list

Reverse and sort list

```
In [66]: list1
```

```
Out[66]: ['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight']
```

```
In [67]: list1.reverse() # Reverse the list
          list1
```

```
Out[67]: ['eight', 'seven', 'six', 'five', 'four', 'three', 'two', 'one']
```

```
In [68]: list1 = list1[::-1] # Reverse the list
          list1
```

```
Out[68]: ['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight']
```

```
In [69]: mylist3 = [9,5,2,99,12,88,34]
          mylist3.sort() # Sort list in ascending order
          mylist3
```

```
Out[69]: [2, 5, 9, 12, 34, 88, 99]
```

```
In [70]: mylist3 = [9,5,2,99,12,88,34]
          mylist3.sort(reverse=True) # Sort list in descending order
          mylist3
```

```
Out[70]: [99, 88, 34, 12, 9, 5, 2]
```

```
In [71]: mylist4 = [88,65,33,21,11,98]
          sorted(mylist4) # Returns a new sorted list and doesn't change original list
```

```
Out[71]: [11, 21, 33, 65, 88, 98]
```

```
In [73]: mylist4
```

```
Out[73]: [88, 65, 33, 21, 11, 98]
```

count

```
In [74]: list10 = ['one', 'two', 'three', 'four', 'one', 'one', 'two', 'three']
```

```
In [75]: list10.count('one') # Number of times item "one" occurred in the list.
```

```
Out[75]: 3
```

```
In [76]: list10.count('two') # Occurrence of item 'two' in the list
```

```
Out[76]: 2
```

```
In [77]: list10.count('four') # Occurrence of item 'four' in the list
```

```
Out[77]: 1
```

All / Any

The `all()` method returns:

True - If all elements in a list are true

False - If any element in a list is false

The `any()` function returns **True** if any element in the list is **True**. If not, `any()` returns **False**.

```
In [78]: L1 = [1,2,3,4,0]
```

```
In [79]: all(L1) # Will Return false as one value is false (Value 0)
```

```
Out[79]: False
```

```
In [80]: any(L1) # Will Return True as we have items in the list with True value
```

```
Out[80]: True
```

```
In [81]: L2 = [1,2,3,4,True,False]
```

```
In [82]: all(L2) # Returns false as one value is false
```

```
Out[82]: False
```

```
In [83]: any(L2) # Will Return True as we have items in the list with True value
```

```
Out[83]: True
```

```
In [84]: L3 = [1,2,3,True]
```

```
In [85]: all(L3) # Will return True as all items in the list are True
```

```
Out[85]: True
```

```
In [87]: any(L3)
```

```
Out[87]: True
```

```
In [ ]:
```